Web Programming: Node.js, Express, React, & Portfolio Project

P.M.H

03/23/26

Part 1: Key Concepts Summary

This set of lecture notes covers the core topics from the Stanford CS193X lecture on NodeJS and Express, then extends into modern React development. By the end of these notes you will understand how server-side JavaScript works, how to build APIs with Express, and how to create interactive frontends with React.

Topics Covered Today

- **NodeJS**: A JavaScript runtime (built on Chrome's V8 engine, written in C++) that executes JavaScript outside the browser.
- **npm**: Node Package Manager — installs third-party libraries (packages) from `https://www.npmjs.com/`.
- **Express**: A lightweight framework on top of Node's built-in `http` module that simplifies server creation, routing, and middleware.
- **localhost**: When you run a server locally, your computer is both the client and the server. You access it at `http://localhost:PORT`.
- **React**: A modern JavaScript library for building user interfaces with reusable components.

Part 2: NodeJS

NodeJS is a JavaScript runtime written in **C++** that can interpret and execute JavaScript outside of the **browser**. Before Node existed, JavaScript could only run inside a web browser. Node changed that by packaging Chrome's V8 engine into a standalone program that can run on any computer.

Key Components of NodeJS

1. **V8 Engine**: The JavaScript **interpreter/compiler** (from Chrome) that NodeJS uses to interpret, compile, and execute JavaScript code. It is the same engine that powers Google Chrome, so the JavaScript you write behaves the same way in both environments.
2. **NodeJS API**: A set of JavaScript **libraries** that are useful for creating server programs (e.g., `fs` for file system access, `http` for creating servers, `path` for working with file paths). These are built-in modules that come with Node.

3. Unlike the browser, Node does **not** have access to the **DOM** APIs (like `document.querySelector`). There is no HTML page, no `window` object, and no `document`.

4. Similarly, browsers do **not** have access to **Node** APIs (like `fs.readFile`). Browsers run in a sandboxed environment and cannot directly touch the file system for security reasons.

Running Node

You can run Node in two ways:

```
# 1. Interactive REPL (Read-Eval-Print Loop)
$ node
> let x = 5;
undefined
> x++
5
> x
6

# 2. Execute a JavaScript file
$ node myScript.js
```

The REPL is useful for quick experiments, similar to the Chrome DevTools console or typing `python` in a terminal. For real projects, you'll write `.js` files and execute them with `node fileName.js`.

Common Gotcha

If you write `document.querySelector('div')` in a Node script, you will get:
`ReferenceError: document is not defined`
The DOM only exists in the **browser**. Node provides server-side APIs instead. This is one of the most common mistakes when first learning Node — remember that Node and the browser are two separate JavaScript environments that share the same language but have different built-in objects.

Node for Servers

The primary use case for Node is building web servers. Node's built-in `http` module lets you create a server that listens for incoming requests:

```
const http = require('http');

const server = http.createServer();

server.on('request', function(req, res) {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello World\n');
});

server.on('listening', function() {
  console.log('Server running!');
});

server.listen(3000);
```

This works, but the raw `http` API is quite low-level. You have to build the request parsing manually, write the response headers manually, and there is a lot of tedious boilerplate code. That's where Express comes in (Part 4).

Part 3: npm — The Node Package Manager

When you install NodeJS, you also get **npm**. npm is a **command-line** tool that lets you install **packages** (libraries and tools) written in JavaScript. You can browse the entire registry of available packages at `https://www.npmjs.com/`.

Essential npm Commands

1. `npm init -y`: Creates a new `package.json` file with default values. This file is the manifest for your project — it tracks your project's name, version, scripts, and most importantly, its dependencies.
2. `npm install express`: Downloads the `express` package and all of its dependencies into a `node_modules` folder. It also adds `express` to the `dependencies` section of your `package.json`.
3. `npm uninstall express`: Removes the `express` package from `node_modules` and removes it from your `package.json` dependencies.
4. `npm install` (no package name): Reads your `package.json` and installs *all* listed dependencies into `node_modules`. This is what collaborators run after cloning your repo.

About `node_modules`

When you run `npm install`, npm downloads packages into a local `node_modules` folder. This folder can be huge — you should **never** commit it to version control. Instead, add it to your `.gitignore`. Anyone who clones your repo can recreate it by running `npm install`, because the `package.json` file records exactly which packages (and which versions) are needed.

Understanding `package.json`

The `package.json` is the heart of any Node project. It serves as a manifest that tells npm (and anyone reading your code) everything about the project. Here is a minimal example:

```
{
  "name": "my-server",
  "version": "1.0.0",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

The `scripts` section lets you define shortcuts. With the above config, running `npm start` in the terminal will execute `node server.js`.

Part 4: Building Servers with Express

The Node `http` module can create servers, but its API is very **low-level** and verbose. **Express** is a lightweight framework that simplifies server creation by providing clean abstractions for routing, middleware, and response handling.

Express is not part of Node's built-in APIs — it's a third-party package. If you try to `require('express')` without installing it first, you'll get: `Error: Cannot find module 'express'`. You must install it via npm first.

Your First Express Server

```
const express = require('express');
const app = express();

const PORT = 3000;

// Define a route: when someone visits '/', respond with text
app.get('/', (req, res) => {
  res.send('Hello World!');
});

// Start the server
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

Let's break down each part of this code:

1. `require('express')` loads the Express **library/module** from `node_modules`.
2. `express()` creates a new Express **application** instance. This object is your server.
3. `app.get('/', ...)` defines a **route** that responds to HTTP GET requests at the root path (`/`).
4. The callback function `(req, res) =>` {...} is the **route handler**. It runs every time someone visits that path.
5. `res.send('Hello World!')` sends a **response** back to the client.
6. `app.listen(3000, ...)` tells the server to start listening on port **3000**.

Compare this to the raw `http` version — Express takes what was 15+ lines of boilerplate and reduces it to about 6 meaningful lines.

Express Routes

Routes follow the pattern: `app.`*`method`*`(`*`path, handler`*`)`

The **method** matches the HTTP verb (`get`, `post`, `put`, `delete`, etc.). The **path** is the URL pattern. The **handler** is a callback function.

Every route handler receives two objects:

- `req` (**Request**): Contains information about the incoming HTTP request — the URL, query parameters, headers, body, etc.
- `res` (**Response**): Provides methods to send a response back to the client — `res.send()`, `res.json()`, `res.status()`, etc.

Query Parameters

When a URL contains query parameters (e.g., `/hello?name=Alice&age=25`), Express automatically parses them into the `req.query` object:

```
app.get('/hello', (req, res) => {
  const name = req.query.name;   // "Alice"
  const age = req.query.age;     // "25"
  res.send(`Hello, ${name}! You are ${age}.`);
});
```

Note that query parameter values are always strings, even if they look like numbers. If you need a number, you must convert it with `parseInt()` or `Number()`.

Serving Static Files

To serve HTML, CSS, and JS files from a folder, use `express.static()`:

```
// Serve all files inside the "public" folder
app.use(express.static('public'));
```

Now if you have `public/index.html`, `public/style.css`, etc., they are automatically served at the root URL. A request to `http://localhost:3000/style.css` will serve the file at `public/style.css`.

Why not open HTML files directly?

If you open an HTML file via `file://` and try to `fetch()` your server, you will get a **CORS error**. This happens because the browser treats `file://` and `http://localhost:3000` as different origins. The solution: serve your frontend files from the *same server* using `express.static()`, so both your page and your API share the same origin.

POST Requests and JSON Bodies

To receive JSON data from the client, you need the `express.json()` middleware:

```
app.use(express.json()); // REQUIRED to parse JSON bodies

app.post('/api/messages', (req, res) => {
  const newMessage = req.body; // parsed JSON object
  // ... save it, process it, etc.
  res.json({ status: 'success', message: newMessage });
});
```

Key Points about POST Requests

- Use `method: 'POST'` in the fetch options.
- Set the `Content-Type` header to `'application/json'`.
- Use `JSON.stringify()` to convert your JS object into a JSON string for the body.
- On the server, `express.json()` middleware is **required** — without it, `req.body` will be `undefined`.

Here is a complete `fetch()` example from the client side:

```
// Client-side code (in the browser)
const response = await fetch('/api/messages', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({ text: 'Hello from the client!' })
});
const data = await response.json();
console.log(data); // { status: 'success', message: {...} }
```

Understanding localhost

When you run your Express server and visit `http://localhost:3000`, your computer is acting as **both the client and the server**. The browser (client) sends a request, and your Node/-Express process (server) receives it and sends back a response — all on the same machine. The

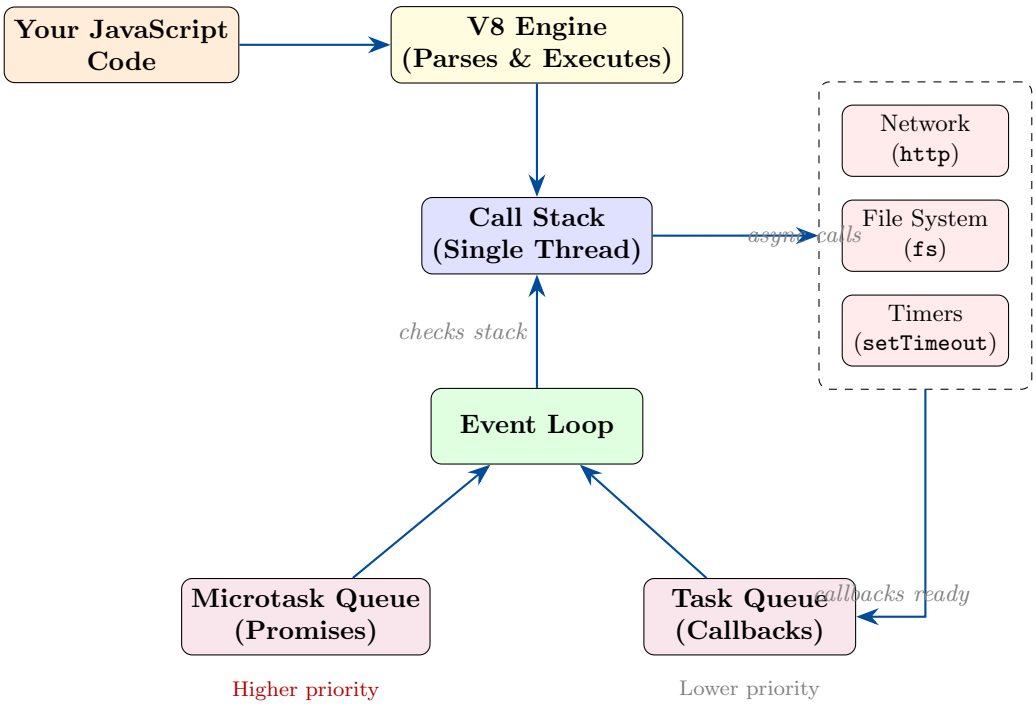
term `localhost` is just a hostname that always points back to your own computer (IP address `127.0.0.1`).

In a production environment, you would deploy your server to a remote machine (like AWS, Heroku, or Render) and replace `localhost` with a real domain name.

Part 5: How Node.js Works

The Node.js Architecture

The following diagram shows how Node.js processes JavaScript using its single-threaded, event-driven model:



How the Event Loop Works (Step by Step)

1. Your JavaScript code is parsed and executed by the **V8 engine**.
2. Synchronous function calls go onto the **call stack** and execute one at a time (single-threaded).
3. When an **async operation** is encountered (network request, file read, timer), it is handed off to the **Node APIs** (written in C++), which run in background threads.
4. When an async operation completes, its callback is placed in the **task queue** (or **micro-task queue** for Promises).
5. The **event loop** continuously checks: "Is the call stack empty?" If yes, it moves the next callback from the queues onto the stack.
6. **Microtasks** (Promises) always have priority over regular tasks (like `setTimeout` callbacks).

Why This Matters

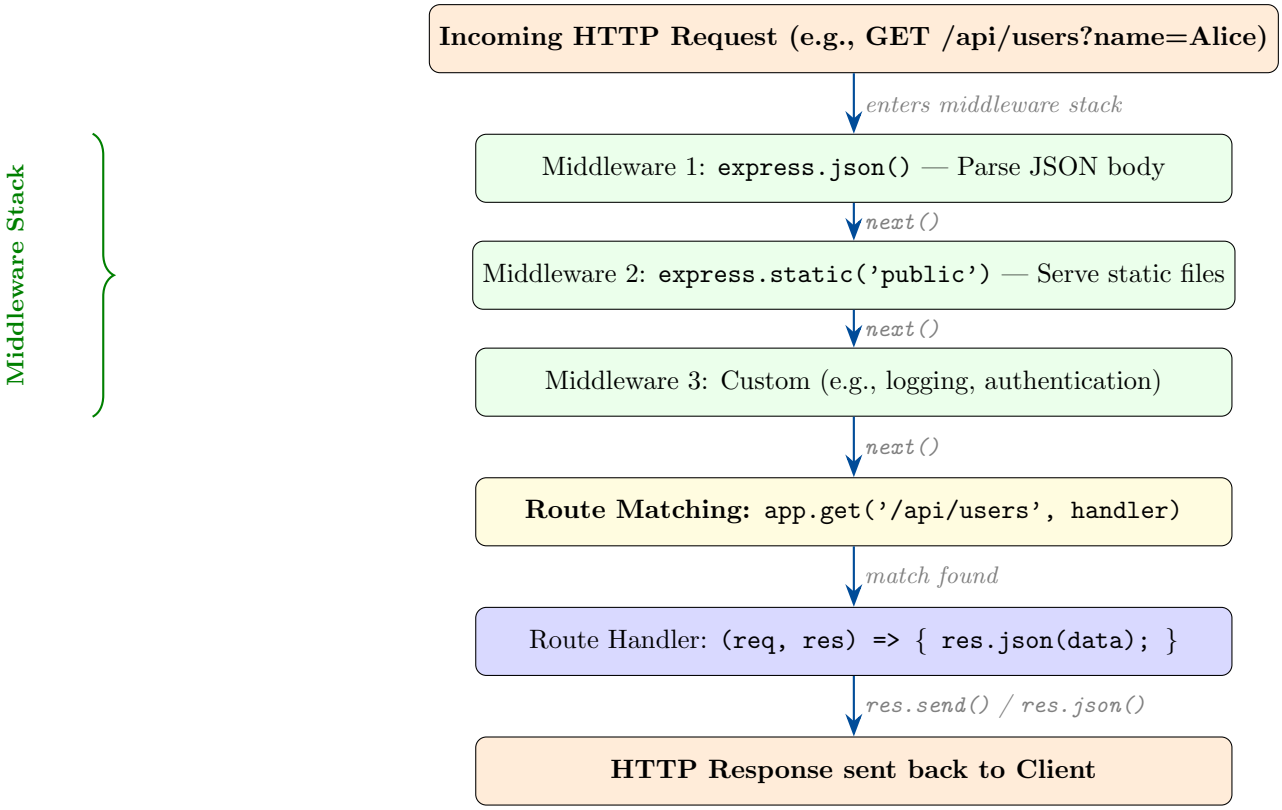
This architecture is why Node.js can handle thousands of simultaneous connections with a single thread. Instead of blocking while waiting for I/O, Node delegates the work and continues processing other requests. This is called **non-blocking I/O**. It is fundamentally different from traditional server models (like Apache) that create a new thread for each connection.

Key Takeaways About the Event Loop

- JavaScript is **single**-threaded: it only runs one piece of code at a time on the call stack.
- The **Node APIs (libuv)** handle async operations like network requests and file reads in background threads managed by the operating system.
- **Microtasks** (from Promises and `async/await`) have higher priority than regular task queue callbacks (from `setTimeout`, I/O callbacks, etc.).
- The **event loop** continuously checks if the call stack is empty, then moves the next task onto it. This is what makes asynchronous JavaScript work.
- If you put a long-running synchronous operation on the call stack (like a `while` loop that runs for 10 seconds), *everything else is blocked* — no callbacks can execute, no new requests can be handled. This is why you should never block the event loop.

How Express Works on Top of Node.js

Express adds a layer of abstraction over Node's raw `http` module. The diagram below shows how an HTTP request flows through an Express application:



Key Concepts in the Express Architecture

1. **Middleware**: Functions that run *before* your route handler. Each middleware can modify `req`/`res` or call `next()` to pass control to the next function in the stack. If a middleware doesn't call `next()`, the request stops there.

2. **Routing**: Express matches the request method (GET, POST, etc.) and URL path to the correct handler. If no route matches, Express returns a 404.

3. **Request object** (`req`): Contains `req.query` (parsed query string), `req.body` (parsed request body), `req.params` (URL parameters like `/users/:id`), headers, cookies, and more.

4. **Response object** (`res`): Provides `res.send()` (send text/HTML), `res.json()` (send JSON), `res.status()` (set HTTP status code), `res.redirect()` (redirect to another URL), and more.

The middleware pattern is powerful because it lets you compose behavior. For example, you can add logging, authentication, CORS headers, and body parsing as separate middleware functions, keeping each one focused on a single responsibility.

Part 6: Introduction to React

Now that you understand how to build a backend with Node.js and Express, let's learn how to build modern, interactive frontends with **React**.

What is React and Why Use It?

React is a JavaScript **library** (created by Facebook/Meta) for building user interfaces. Instead of manipulating the DOM directly (like we did with `document.querySelector` and `document.createElement`), React lets you build your UI out of reusable **components**.

With vanilla JavaScript, updating the UI means manually finding DOM elements, changing their content, adding/removing classes, etc. As an app grows, this becomes extremely difficult to manage. React solves this by letting you *describe* what the UI should look like for any given state, and it automatically figures out what DOM changes are needed.

Why React?

- **Component-based**: Break your UI into small, reusable pieces. Each component manages its own state and rendering logic.
- **Declarative**: You describe *what* the UI should look like, and React figures out *how* to update the DOM. No more `element.innerHTML = ...` spaghetti.
- **Efficient**: React uses a "virtual DOM" to minimize expensive real DOM operations. It calculates the minimal set of changes needed and applies them in a batch.
- **Industry standard**: Used by Facebook, Instagram, Netflix, Airbnb, and thousands of other companies. It is the most popular frontend framework/library.

Setting Up a React Project

The easiest way to create a new React project is with Vite (a fast, modern build tool):

```
# Create a new React project
$ npm create vite@latest my-portfolio -- --template react

# Navigate into the project and install dependencies
$ cd my-portfolio
$ npm install

# Start the development server
$ npm run dev
```

This gives you a project with hot-reloading — every time you save a file, the browser updates automatically. Vite is significantly faster than older tools like Create React App because it uses native ES modules during development.

Components: The Building Blocks of React

In React, everything is a **component**. A component is just a JavaScript function that returns **JSX** (HTML-like syntax inside JavaScript):

```
// A simple functional component
function Greeting() {
  return (
    <div>
      <h1>Hello, World!</h1>
      <p>Welcome to my website.</p>
```

```
    </div>
  );
}

export default Greeting;
```

This looks like HTML, but it's actually JSX — a syntax extension that gets compiled into regular JavaScript function calls. The code above is equivalent to writing `React.createElement('div', null, React.createElement('h1', null, 'Hello, World!'), ...)` but much more readable.

JSX Rules to Remember

- JSX looks like HTML but it's actually JavaScript. It gets compiled by Babel/Vite before running in the browser.
- Every component must return a **single parent element** (wrap in `<div>` or use a Fragment: `<>...</>`).
- Use `className` instead of `class` (because `class` is a reserved word in JS).
- JavaScript expressions go inside curly braces: `{variable}`, `{2 + 2}`, `{items.map(...)}`.
- Self-closing tags must have a slash: `<img />`, `<br />`, `<Component />`.

Props: Passing Data to Components

Props (short for "properties") let you pass data from a parent component to a child component, just like HTML attributes:

```
// Parent passes data via attributes
function App() {
  return (
    <div>
      <Greeting name="Alice" />
      <Greeting name="Bob" />
    </div>
  );
}

// Child receives data via the props parameter
function Greeting({ name }) {
  return <h1>Hello, {name}!</h1>;
}
```

Props are **immutable** (read-only). A child component can never modify its own props — data flows *one way*, from parent to child. This is called **one-way data flow** and it makes your application much easier to reason about, because you always know where data is coming from.

useState: Adding Interactivity

The `useState` hook lets your component "remember" values between renders. This is how you make interactive components:

```
import { useState } from 'react';

function Counter() {
  // Declare a state variable "count" with initial value 0
  const [count, setCount] = useState(0);
```

```

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}
```

Here's how `useState` works:

1. `useState(0)` returns an array with two items: the current **state value** (here, `count`) and a **setter function** (here, `setCount`) to update it. The `0` is the initial value.
2. When you call `setCount(count + 1)`, React **re-renders** the component with the new value. The entire function runs again, but this time `count` returns the updated value.
3. You should **never** modify state directly (e.g., `count = count + 1`). Always use the **setter** function. Direct mutation doesn't trigger a re-render, so the UI won't update.
4. You can have multiple `useState` calls in a single component — one for each piece of state you need to track.

useEffect: Side Effects (Brief Mention)

Sometimes you need to do something *after* your component renders — like fetching data from an API, setting up a timer, or updating the document title. That's what `useEffect` is for:

```
import { useState, useEffect } from 'react';

function UserProfile({ userId }) {
  const [user, setUser] = useState(null);

  useEffect(() => {
    // This runs after the component renders
    fetch(`/api/users/${userId}`)
      .then(res => res.json())
      .then(data => setUser(data));
  }, [userId]); // Re-run only when userId changes

  if (!user) return <p>Loading...</p>;
  return <h1>{user.name}</h1>;
}
```

useEffect in a Nutshell

- `useEffect(() => {...}, [])` — runs **once** after the first render (empty dependency array). Good for initial data fetching.
- `useEffect(() => {...}, [x])` — runs after the first render **and** whenever `x` changes. Good for reacting to prop/state changes.
- `useEffect(() => {...})` — runs after **every** render (no dependency array). Rarely what you want.
- Think of it as: "after this component renders, also do this side effect."

We will cover `useEffect` in much more detail in a future lecture. For now, just know it exists and that it's how you connect React components to external data sources.

Part 7: Barebone React Portfolio Website

Below is a complete, minimal React portfolio website. It uses **React Router** for client-side navigation and includes a Home/Hero section, About, Projects, Contact, and a `/blog` route. You will use this as the starting point for your homework assignment.

Step 1: Create the Project and Install Dependencies

```
$ npm create vite@latest my-portfolio -- --template react
$ cd my-portfolio
$ npm install
$ npm install react-router-dom
```

React Router (`react-router-dom`) enables client-side routing — navigating between "pages" without a full page reload. The browser URL changes, but React just swaps out which component is displayed.

Step 2: Main Entry Point — `src/main.jsx`

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);
```

The `BrowserRouter` component wraps your entire app and enables React Router's navigation features. `React.StrictMode` is a development helper that warns you about potential problems.

Step 3: App Component with Routing — `src/App.jsx`

```
import { Routes, Route, Link } from 'react-router-dom';
import Home from './pages/Home';
import Blog from './pages/Blog';

function App() {
  return (
    <div className="app">
      <nav className="navbar">
        <Link to="/" className="nav-logo">MyPortfolio</Link>
        <div className="nav-links">
          <a href="#about">About</a>
          <a href="#projects">Projects</a>
          <a href="#contact">Contact</a>
          <Link to="/blog">Blog</Link>
        </div>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/blog" element={<Blog />} />
```

```
      </Routes>
    </div>
  );
}

export default App;
```

Notice the difference between `<Link>` and `<a>`: the `Link` component from React Router performs client-side navigation (no page reload), while `<a href="#about">` scrolls to an element on the same page using a hash link.

Step 4: Home Page — `src/pages/Home.jsx`

```
function Home() {
  return (
    <main>
      {/* Hero Section */}
      <section className="hero">
        <h1>Hi, I'm [Your Name]</h1>
        <p>Aspiring web developer & creative thinker</p>
      </section>

      {/* About Section */}
      <section id="about" className="section">
        <h2>About Me</h2>
        <p>
          I'm a student learning web development. I love
          building things with code and exploring new
          technologies. Currently studying at [University].
        </p>
      </section>

      {/* Projects Section */}
      <section id="projects" className="section">
        <h2>Projects</h2>
        <div className="projects-grid">
          <ProjectCard
            title="Project 1"
            desc="A cool web app built with React."
          />
          <ProjectCard
            title="Project 2"
            desc="An Express API for managing tasks."
          />
          <ProjectCard
            title="Project 3"
            desc="A portfolio website (this one!)."
          />
        </div>
      </section>

      {/* Contact Section */}
      <section id="contact" className="section">
        <h2>Contact</h2>
        <p>Email: yourname@example.com</p>
        <p>GitHub: github.com/yourusername</p>
      </section>
    </main>
  );
}

function ProjectCard({ title, desc }) {
```

```
  return (
    <div className="project-card">
      <h3>{title}</h3>
      <p>{desc}</p>
    </div>
  );
}

export default Home;
```

Notice how `ProjectCard` is a reusable component that accepts `title` and `desc` props. Instead of copy-pasting the same HTML three times, we define the structure once and reuse it with different data. This is the power of component-based architecture.

Step 5: Blog Page — `src/pages/Blog.jsx`

```
function Blog() {
  return (
    <main className="section">
      <h2>My Blog</h2>
      <p>Welcome to my blog! This is where I'll document
         my spring break adventures.</p>

      <article className="blog-post">
        <h3>Spring Break 2026</h3>
        <p>Coming soon... check back after break!</p>
      </article>
    </main>
  );
}

export default Blog;
```

Step 6: Basic Styles — `src/index.css`

```
* { box-sizing: border-box; margin: 0; padding: 0; }

body {
  font-family: 'Segoe UI', Tahoma, sans-serif;
  line-height: 1.6;
  color: #333;
}

.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1rem 2rem;
  background: #1a1a2e;
  color: #fff;
  position: sticky;
  top: 0;
}

.nav-logo {
  font-size: 1.4rem;
  font-weight: bold;
  color: #e94560;
  text-decoration: none;
}

```

```
.nav-links a {
  color: #fff;
  text-decoration: none;
  margin-left: 1.5rem;
}

.nav-links a:hover { color: #e94560; }

.hero {
  text-align: center;
  padding: 6rem 2rem;
  background: linear-gradient(135deg, #0f3460, #16213e);
  color: #fff;
}

.hero h1 { font-size: 2.8rem; margin-bottom: 0.5rem; }
.hero p  { font-size: 1.2rem; opacity: 0.8; }

.section {
  max-width: 900px;
  margin: 0 auto;
  padding: 4rem 2rem;
}

.section h2 {
  font-size: 2rem;
  margin-bottom: 1rem;
  color: #0f3460;
}

.projects-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 1.5rem;
  margin-top: 1rem;
}

.project-card {
  border: 1px solid #ddd;
  border-radius: 8px;
  padding: 1.5rem;
  transition: box-shadow 0.2s;
}

.project-card:hover {
  box-shadow: 0 4px 12px rgba(0,0,0,0.1);
}

.project-card h3 { color: #e94560; margin-bottom: 0.5rem; }

.blog-post {
  border-left: 4px solid #e94560;
  padding: 1rem 1.5rem;
  margin-top: 1.5rem;
  background: #f9f9f9;
  border-radius: 0 8px 8px 0;
}
```

Step 7: Run It!

```
$ npm run dev
```

Open the URL shown in the terminal (usually `http://localhost:5173`) to see your portfolio!

Part 8: Homework Assignment — React Portfolio & Spring Break Blog

Assignment Overview

Using the barebone portfolio from Part 7 as your starter code, build a **professional-looking portfolio website** using React. Deploy it to GitHub Pages and add a blog sub-page documenting your spring break.

Requirements

1. **Professional Portfolio** — Customize the starter code into a polished, personal portfolio:
   - Replace all placeholder text with your real information.
   - Add custom styling (colors, fonts, layout) to make it your own.
   - Include at least 3 project cards with descriptions (can be class projects, personal projects, or ideas).
   - Make it responsive (looks good on both desktop and mobile).
2. **Spring Break Blog** (`/blog` route) — Document your spring break:
   - Include **photos and/or videos** of your vacation (use `<img>` and `<video>` tags or embed links).
   - Write about your experience like a **real travel blog entry** — not just a list of images. Include storytelling, descriptions of places, food, activities, and reflections.
   - At least 2–3 paragraphs of written content per blog post.
3. **Deploy to GitHub Pages** — Follow the steps below to host your site live on the internet.

Step-by-Step: Deploying to GitHub Pages

Step 1: Create a GitHub repository:

```
# Initialize git in your project (if not already)
$ git init
$ git add .
$ git commit -m "Initial portfolio commit"

# Create a repo on GitHub, then push
$ git remote add origin https://github.com/YOUR_USERNAME/my-portfolio.git
$ git branch -M main
$ git push -u origin main

```

Step 2: Install the GitHub Pages deployment package:

```
$ npm install --save-dev gh-pages

```

Step 3: Update `vite.config.js`:

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
```

```
  base: '/my-portfolio/', // Must match your repo name!
});

```

Step 4: Update `package.json` — add these scripts:

```
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "preview": "vite preview",
  "predeploy": "npm run build",
  "deploy": "gh-pages -d dist"
}

```

Step 5: Update your Router to use the base path. In `src/main.jsx`:

```
<BrowserRouter basename="/my-portfolio">
  <App />
</BrowserRouter>

```

Step 6: Deploy!

```
$ npm run deploy

```

Your site will be live at: `https://YOUR_USERNAME.github.io/my-portfolio/`

Step 7: Enable GitHub Pages in your repository settings:

- Go to your repo on GitHub → Settings → Pages.
- Under "Source," select the `gh-pages` branch.
- Click Save. Your site will be live within a few minutes.

Grading Criteria

Rubric (100 points)

1. **Design Quality (25 pts)**: Professional appearance, consistent styling, responsive layout, good color/typography choices.
2. **React Usage (25 pts)**: Proper use of components, props, and state. Code is clean and well-organized across multiple files.
3. **GitHub Pages Deployment (25 pts)**: Site is live and accessible at your `github.io` URL. All pages and routes work correctly when deployed.
4. **Blog Content (25 pts)**: The `/blog` page has real spring break content with photos/videos and thoughtful written entries (not just placeholder text or a gallery of images).

Due Date

This assignment is due **the first Thursday after spring break**. Submit the GitHub Pages URL on Canvas.